



南昌大学

大作业报告

课程名： WEB 程序设计

学 院： 数学与计算机 系 计算机科学与技术系

专 业： 数据科学与大数据技术

班 级： 数据科学与大数据技术 231 班

姓 名： 郭峰

项目名称： PP Chat 在线聊天系统

任课教师： 刘涛, 万明

授课学期： 2025 年 ~ ~ ~ 2026 年 2 学期

心得体会

在本次《WEB 程序设计》大作业中，我主要负责语音消息模块，同时也参与了项目统筹、联调和部署过程。相比普通的登录、增删改查功能，我认为语音模块更有挑战性，因为它涉及浏览器端音频采集、WebSocket 实时传输、后端文件处理和前端播放展示等多个环节，任何一个环节出问题，最终功能都无法真正跑通。

一开始我对语音消息的理解比较简单，觉得“录完音上传一下就行”。但在真正实现时，我很快发现语音消息并不是一个孤立功能，而是必须嵌入现有聊天架构中。如果单独做成一个上传接口，会和现有文本消息链路割裂，前端也要额外处理很多状态。后来我逐渐意识到，更合理的做法是把语音消息视为“消息的一种类型”，让它和文本消息共用同一套 WebSocket 通道、数据库结构和消息渲染逻辑。这个思路的变化，对我理解系统设计帮助很大。

在实现过程中，我遇到的第一个明显问题是浏览器录音。文本消息只需要读取输入框内容，但语音消息必须先向浏览器申请麦克风权限，再通过 `MediaRecorder` 获取音频流。最初项目直接部署到服务器上时并没有配置 HTTPS，结果浏览器会因为安全策略限制，拒绝在非安全上下文中调用麦克风功能，这导致语音录制根本无法正常使用。后来在补上 `Nginx` 反向代理和 HTTPS 证书之后，麦克风权限问题才真正解决。这个过程让我很直观地认识到，某些前端功能不只是代码写对就行，还依赖部署环境是否满足浏览器的安全要求。除此之外，我还发现“录下来以后怎么处理”同样重要。例如录音过短时没有意义，录音过长又可能导致传输过慢，所以最终增加了最短和最长时长控制，并且在界面上给出倒计时提示。这让我体会到，好的功能实现不只是底层代码跑通，还要兼顾交互体验。

第二个让我印象很深的问题是消息体积控制。最开始我尝试直接把语音消息用 `Base64` 编码后存放在 `MySQL`，虽然实现简单，但很快就发现数据库膨胀得非常明显：语音本身就是大对象，再经过 `Base64` 编码后体积会进一步增大，导致数据库记录非常笨重，查询、导出和后台查看都会受影响。后来我调整思路，不再把完整 `Base64` 原文长期保存在数据库，而是改成“文件落盘、数据库只存 URL”的方案。与此同时，在实际发送文件的过程中我还发现，如果录音时长过长，消息发送容易出现不完整、传输失败或处理超时等问题。为了解决这个问题，最终将单条语音时长限制为 10 秒，这样既控制了消息大小，也提高了发送成功率和播放体验。这个过程让我认识到：很多线上问题并不是“代码语法错了”，而是数据

体积、传输方式和运行环境之间没有平衡好。

此外，在项目进入后期后，我还参与了线上部署。将项目真正部署到服务器、绑定域名、配置 Nginx、申请 HTTPS 证书，这些步骤和本地开发完全不同。特别是在证书申请时，因为 80 端口没有放开，导致 Certbot 验证超时。这个问题让我明白，项目从“代码完成”到“可被访问”之间，还有完整的一层部署与运维工作。以前做实验常常只停留在本地运行成功，这次则更真实地接触到了完整交付流程。

总体来说，这次大作业让我最大的收获不是单一学会了某个 API，而是更深刻地理解了一个完整 Web 项目是如何从需求、设计、编码、联调、测试一路走到部署上线的。语音模块虽然只是六个分工模块中的一个，但它横跨前端、后端、实时通信和文件存储四个方面，对我锻炼非常大。通过这次实践，我对 Spring Boot 项目结构、WebSocket 通信流程、文件上传处理以及系统部署都有了更扎实的认识。以后如果再做类似项目，我会更加重视前期设计和后期联调，也会更主动地从整体角度思考问题，而不是只盯着某一段代码。